

## **OBJETIVOS**

- Reconocer las ventajas y aplicaciones del control de eventos en JavaScript
- Identificar los principales métodos de captura de eventos
- Asimilar el funcionamiento de los eventos y su propagación por los elementos del DOM
- Crear aplicaciones capaces de lanzar código asociado a eventos
- Reconocer las aplicaciones del objeto de evento y sus principales métodos y propiedades
- Diferenciar los principales tipos de eventos identificando sus acciones principales
- Aplicar el uso eventos a la interacción con el usuario utilizando formularios
- Resolver problemas de aplicaciones web completas utilizando control de eventos

## **CONTENIDOS**

### **8.1 INTRODUCCIÓN A LOS EVENTOS**

### **8.2 CAPTURA DE EVENTOS**

- 8.2.1 MÉTODOS CLÁSICOS Y OBSOLETOS
- 8.2.2 MÉTODO ACONSEJABLE
- 8.2.3 PROPAGACIÓN DE EVENTOS
- 8.2.4 ANULAR EVENTOS
- 8.2.5 CAPTURA DE EVENTOS EN ELEMENTOS DINÁMICOS

### **8.3 OBJETO DE EVENTO**

- 8.3.1 UTILIDAD Y USO DEL OBJETO DE EVENTO
- 8.3.2 OBTENER COORDENADAS DEL EVENTO
- 8.3.3 OBTENER LA TECLA PULSADA
- 8.3.4 ANULAR COMPORTAMIENTOS PREDETERMINADOS
- 8.3.5 CANCELAR PROPAGACIÓN
- 8.3.6 LANZAR EVENTOS

### **8.4 LISTA DE TIPOS DE EVENTOS**

- 8.4.1 EVENTOS DE RATÓN
- 8.4.2 EVENTOS DE TECLADO
- 8.4.3 EVENTOS DE MOVIMIENTO EN LA VENTANA
- 8.4.4 EVENTOS SOBRE CARGA Y DESCARGA DE ELEMENTOS
- 8.4.5 OTROS EVENTOS

### **8.5 FORMULARIOS**

- 8.5.1 EL FORMULARIO COMO OBJETO DEL DOM
- 8.5.2 EVENTOS DE FORMULARIO
- 8.5.3 PROPIEDADES DE LOS CONTROLES
- 8.5.4 MÉTODOS DE LOS CONTROLES

## 8.1 Introducción a los eventos

### Qué son los eventos en JavaScript

En JavaScript, un **evento** es una acción que ocurre en el navegador, generalmente como resultado de la interacción del usuario con la página web. Un evento puede ser un clic, mover el ratón, presionar una tecla, o incluso cuando una imagen se carga completamente.

### ¿Cómo manejar eventos en JavaScript?

Para **manejar un evento** en JavaScript, necesitamos asignar un **gestor de eventos** a un elemento del DOM. Esto se hace mediante el uso de la función `addEventListener()`, que permite asociar un evento con un elemento específico.

#### Sintaxis básica de `addEventListener`:

```
element.addEventListener('evento', funcion);
```

- `element`: el elemento del DOM al que se le agregará el evento (por ejemplo, un botón, una imagen, etc.).
- `'evento'`: el tipo de evento que se desea manejar, como `'click'`, `'mouseover'`, `'keydown'`, etc.
- `funcion`: la función que se ejecutará cuando el evento ocurra.

#### Ejemplo básico:

```
<button id="miBoton">Haz clic aquí</button>
<script>
  let boton = document.getElementById('miBoton');

  boton.addEventListener('click', function() {
    alert('¡Botón clickeado!');
  });
</script>
```

Este código muestra una alerta cuando el usuario hace clic en el botón.

## Ejercicios propuestos:

### Ejercicio 1:

**Objetivo:** Crear un botón que, al hacer clic, muestre un mensaje en la consola.

#### Instrucciones:

1. Crea un botón en HTML con el texto "Haz clic aquí".
2. Utiliza `addEventListener` para que, cuando el botón sea clickeado, se muestre un mensaje en la consola que diga "¡Evento de clic activado!".

#### Código de ejemplo:

```
<button id="boton">Haz clic aquí</button>
<script>
  let boton = document.getElementById('boton');

  boton.addEventListener('click', function() {
    console.log('¡Evento de clic activado!');
  });
</script>
```

### Ejercicio 2:

**Objetivo:** Mostrar un mensaje de alerta al pasar el ratón sobre un área definida.

#### Instrucciones:

1. Crea una caja (`div`) en HTML con un fondo de color.
2. Usa el evento `'mouseover'` para que cuando el ratón pase sobre la caja, aparezca un mensaje de alerta.

#### Código de ejemplo:

```
<div id="miCaja" style="width: 200px; height: 200px; background-color:
lightblue;"></div>
<script>
  let caja = document.getElementById('miCaja');

  caja.addEventListener('mouseover', function() {
    alert('¡Estás sobre la caja!');
  });
</script>
```

### Ejercicio 3:

**Objetivo:** Cambiar el color de fondo de un elemento al hacer clic.

#### Instrucciones:

1. Crea un elemento `div` con un fondo de color inicial.
2. Al hacer clic en el `div`, cambia el color de fondo a un nuevo color.

#### Código de ejemplo:

```
<div id="cajaColor" style="width: 200px; height: 200px; background-color: red;"></div>
<script>
  let cajaColor = document.getElementById('cajaColor');

  cajaColor.addEventListener('click', function() {
    cajaColor.style.backgroundColor = 'green';
  });
</script>
```

### Ejercicio 4:

**Objetivo:** Crear un contador de clics.

#### Instrucciones:

1. Crea un botón en HTML.
2. Cada vez que el usuario haga clic en el botón, aumenta un contador en pantalla.

#### Código de ejemplo:

```
<button id="contadorBoton">Haz clic aquí</button>
<p id="contador">Clicks: 0</p>
<script>
  let boton = document.getElementById('contadorBoton');
  let contador = document.getElementById('contador');
  let cantidad = 0;

  boton.addEventListener('click', function() {
    cantidad++;
    contador.textContent = 'Clicks: ' + cantidad;
  });
</script>
```

## Ejercicio 5:

**Objetivo:** Detectar la tecla presionada por el usuario.

### Instrucciones:

1. Crea un área de texto (`textarea`) en HTML.
2. Cuando el usuario presione una tecla, muestra un mensaje con el código de la tecla.

### Código de ejemplo:

```
<textarea id="miAreaTexto" rows="4" cols="50"></textarea>
<script>
  let areaTexto = document.getElementById('miAreaTexto');

  areaTexto.addEventListener('keydown', function(event) {
    console.log('Tecla presionada: ' + event.key);
  });
</script>
```

## Soluciones de los ejercicios de introducción a los eventos

### 1. Ejercicio 1:

#### Solución:

Al hacer clic en el botón, se muestra el mensaje en la consola:

```
¡Evento de clic activado!.
```

### 2. Ejercicio 2:

#### Solución:

Al pasar el ratón sobre la caja, aparecerá un mensaje de alerta:

```
¡Estás sobre la caja!.
```

### 3. Ejercicio 3:

#### Solución:

Al hacer clic en el `div`, el color de fondo cambiará de rojo a verde.

### 4. Ejercicio 4:

#### Solución:

El contador aumentará cada vez que se haga clic en el botón.

### 5. Ejercicio 5:

#### Solución:

El código de la tecla presionada se imprimirá en la consola cada vez que el usuario presione una tecla en el área de texto.

## Resumen de Introducción a los eventos

En esta sección hemos aprendido cómo utilizar los eventos en JavaScript para hacer que nuestra página web sea más interactiva. Los eventos permiten que una página responda a las acciones del usuario, como clics, teclas presionadas o movimientos del ratón. Para manejar estos eventos, se utiliza el método `addEventListener()`.

En los ejercicios prácticos, aprendiste a:

- Crear eventos básicos como clics y movimientos del ratón.
- Cambiar propiedades de elementos mediante eventos.
- Trabajar con eventos de teclado.

## 8.2 CAPTURA DE EVENTOS

La **captura de eventos** se refiere a **cómo JavaScript detecta y gestiona los eventos** que ocurren sobre los elementos del DOM (clics, teclas, envíos de formularios, etc.).

### 8.2.1 Métodos clásicos y obsoletos

#### Explicación

Antes de `addEventListener`, los eventos se gestionaban de forma **inline** o mediante **propiedades del elemento**.

Estos métodos **funcionan**, pero **no se recomiendan** porque:

- Sobrescriben eventos anteriores
- Mezclan HTML y JavaScript
- Son poco flexibles

#### Métodos clásicos (NO recomendados)

##### 1 Evento inline en HTML

```
<button onclick="saludar()">Saludar</button>

<script>
  function saludar() {
    alert("Hola");
  }
</script>
```

## 2 Asignación directa desde JavaScript

```
boton.onclick = function () {  
    alert("Hola");  
};
```

### Ejercicios propuestos

#### Ejercicio 1

Crea un botón que muestre un `alert` usando un evento inline.

#### Ejercicio 2

Crea un botón y asigna el evento `onclick` desde JavaScript.

#### Ejercicio 3

Comprueba qué ocurre si asignas dos veces `onclick` al mismo botón.

### Soluciones

#### Solución Ejercicio 1

```
<button onclick="alert('Hola')">Saludar</button>
```

#### Solución Ejercicio 2

```
<button id="btn">Pulsar</button>  
<script>  
    document.getElementById("btn").onclick = function () {  
        alert("Botón pulsado");  
    };  
</script>
```

#### Solución Ejercicio 3

```
btn.onclick = function () {  
    console.log("Primero");  
};  
  
btn.onclick = function () {  
    console.log("Segundo");  
};
```

Solo se ejecuta **el segundo**, el primero se pierde.

## 8.2.2 Método aconsejable (addEventListener)

### Explicación

`addEventListener` es el **método recomendado** porque:

- Permite varios eventos en un mismo elemento
- No sobrescribe eventos
- Separa HTML y JavaScript

### Sintaxis

```
element.addEventListener("evento", funcion);
```

### Ejemplo

```
boton.addEventListener("click", function(e) {  
    alert("Has hecho clic");  
});
```

### Ejercicios propuestos

#### Ejercicio 1

Crea un botón que muestre un mensaje al hacer clic usando `addEventListener`.

#### Ejercicio 2

Añade dos eventos `click` al mismo botón y comprueba que ambos se ejecutan.

#### Ejercicio 3

Cambia el texto de un párrafo al pulsar un botón.

### Soluciones

#### Solución Ejercicio 1

```
boton.addEventListener("click", (e) => {  
    alert("Clic detectado");  
});
```

#### Solución Ejercicio 2

```
boton.addEventListener("click", (e) => {  
    console.log("Evento 1");  
});  
boton.addEventListener("click", (e) => {  
    console.log("Evento 2");  
});
```



### Solución Ejercicio 3

```
boton.addEventListener("click", () => {  
    parrafo.textContent = "Texto cambiado";  
});
```

## 8.2.3 Propagación de eventos

### Explicación

Los eventos **se propagan** por el DOM en dos fases principales:

- **Captura** (de arriba hacia abajo)
- **Burbujeo (Bubbling)** (de abajo hacia arriba, la más común)

Por defecto, JavaScript usa **burbujeo (bubbling)**.

### Ejemplo

```
<div id="padre">  
    <button id="hijo">Pulsar</button>  
</div>  
document.getElementById("padre").addEventListener("click", (e) => {  
    console.log("Padre");  
});  
  
document.getElementById("hijo").addEventListener("click", (e) => {  
    console.log("Hijo");  
});
```

Resultado al pulsar el botón: (efecto del bubbling por defecto)

Hijo  
Padre

### Ejercicios propuestos

#### Ejercicio 1

Crea dos elementos anidados y muestra mensajes al hacer clic.

#### Ejercicio 2

Identifica el orden de ejecución.

#### Ejercicio 3

Prueba el tercer parámetro de `addEventListener` (`true`).

## Soluciones

### Solución Ejercicio 3

```
padre.addEventListener("click", (e) => {  
  console.log("Padre captura");  
}, true);
```

Ahora el padre se ejecuta **antes**. (atención al tercer parámetro de `addEventListener`)

## 8.2.4 Anular eventos

### Explicación

Algunos elementos tienen un **comportamiento por defecto**:

- Enlaces → navegar
- Formularios → enviar

Para evitarlo:

```
event.preventDefault();
```

### Ejercicios propuestos

#### Ejercicio 1

Evita que un enlace navegue.

#### Ejercicio 2

Evita el envío de un formulario.

## Soluciones

### Solución Ejercicio 1

```
enlace.addEventListener("click", (e) => {  
  e.preventDefault();  
  alert("Navegación cancelada");  
});
```

### Solución Ejercicio 2

```
form.addEventListener("submit", (e) => {  
  e.preventDefault();  
  alert("Formulario no enviado");  
});
```

## 8.2.5 Captura de eventos en elementos dinámicos

### Explicación

Los elementos creados **después de cargar la página** no tienen eventos asignados.  
Solución: **delegación de eventos**. (Importante porque se usa mucho)

Se asigna el evento a un elemento padre.

### Ejemplo

```
lista.addEventListener("click", (e) => {  
  if (e.target.tagName === "LI") {  
    alert("Elemento pulsado");  
  }  
});
```

### Ejercicios propuestos

#### Ejercicio 1

Crea una lista y detecta clics en sus elementos mostrando el nombre de elemento en consola.

#### Ejercicio 2

Añade elementos dinámicamente y comprueba que funcionan al hacer click en esos nuevos elementos creados.

#### Ejercicio 3

Muestra el texto del elemento pulsado.

### Soluciones

#### Solución Ejercicio 3

```
lista.addEventListener("click", (e) => {  
  alert(e.target.textContent);  
});
```

## Ejercicios propuestos

1. Crea un menú donde cada opción muestre su nombre al hacer clic.
2. Evita que un formulario se envíe si un campo está vacío.
3. Usa propagación para detectar clics en una tabla.
4. Implementa delegación de eventos en una lista dinámica.
5. Combina `preventDefault()` y `stopPropagation()`.

### Solución ejemplo (Ejercicio 2)

```
form.addEventListener("submit", (e) => {  
  if (input.value === "") {  
    e.preventDefault();  
    alert("Campo obligatorio");  
  }  
});
```

## Resumen de Captura de Eventos

Métodos antiguos vs modernos

Uso correcto de `addEventListener`

Propagación de eventos

Cancelación de acciones

Delegación de eventos en elementos creados dinámicamente

## 8.3 OBJETO DE EVENTO

El **objeto de evento** es un objeto que JavaScript crea automáticamente cuando ocurre un evento.

Este objeto contiene **información clave** sobre lo que ha pasado:

- qué elemento lo provocó ( origen del evento – `event.target` || `event.srcElement` )
- qué tecla se pulsó. ( si evento de teclado )
- dónde ocurrió el clic (si evento de ratón)
- etc.

### 8.3.1 Utilidad y uso del objeto de evento

#### Explicación

Cuando se ejecuta un evento, la función asociada recibe un **parámetro** (normalmente llamado `event` o **e**).

```
element.addEventListener("click", function (event) {  
    console.log(event);  
});
```

Gracias a este objeto podemos:

- Saber **qué elemento** lanzó el evento (`event.target`)
- Evitar acciones por defecto
- Cancelar propagación
- Obtener información del teclado o ratón

#### Ejemplo sencillo

```
boton.addEventListener("click", function (e) {  
    console.log("Elemento pulsado:", e.target);  
});
```

#### Ejercicios propuestos

##### Ejercicio 1

Muestra en consola el objeto `event` al hacer clic en un botón.

##### Ejercicio 2

Muestra el nombre de la etiqueta HTML que ha lanzado el evento.

### Ejercicio 3

Cambia el color del elemento que se ha pulsado.

## Soluciones

### Solución Ejercicio 1

```
boton.addEventListener("click", (e) => {  
    console.log(e);  
});
```

### Solución Ejercicio 2

```
boton.addEventListener("click", (e) => {  
    console.log(e.target.tagName);  
});
```

### Solución Ejercicio 3

```
boton.addEventListener("click", (e) => {  
    e.target.style.backgroundColor = "yellow";  
});
```

## 8.3.2 Obtener coordenadas del evento

### Explicación

En eventos de ratón podemos obtener **coordenadas** del evento:

- `event.clientX` → posición X
- `event.clientY` → posición Y

Son muy útiles para juegos, dibujos, menús contextuales, etc.

### Ejemplo

```
document.addEventListener("click", (e) => {  
    console.log("X:", e.clientX, "Y:", e.clientY);  
});
```

## Ejercicios propuestos

### Ejercicio 1

Muestra las coordenadas del clic en consola.

### Ejercicio 2

Muestra las coordenadas dentro de un párrafo.

### Ejercicio 3

Cambia la posición de un div al hacer clic.

## Soluciones

### Solución Ejercicio 1

```
document.addEventListener("click", (e) => {  
  console.log(e.clientX, e.clientY);  
});
```

### Solución Ejercicio 2

```
document.addEventListener("click", (e) => {  
  parrafo.textContent = `X: ${e.clientX}, Y: ${e.clientY}`;  
});
```

### Solución Ejercicio 3

```
document.addEventListener("click", (e) => {  
  caja.style.left = e.clientX + "px";  
  caja.style.top = e.clientY + "px";  
});
```

## 8.3.3 Obtener la tecla pulsada

### Explicación

En eventos de teclado usamos:

- `event.key` → tecla pulsada
- `keydown`, `keyup` (eventos producidos - este es el nombre del evento)

### Ejemplo

```
document.addEventListener("keydown", (e) => {  
  console.log("Tecla:", e.key);  
});
```

## Ejercicios propuestos

### Ejercicio 1

Muestra la tecla pulsada.

### Ejercicio 2

Detecta si se pulsa la tecla Enter.

### Ejercicio 3

Cambia el fondo al pulsar una tecla concreta.

## Soluciones

### Solución Ejercicio 1

```
document.addEventListener("keydown", (e) => {  
    console.log(e.key);  
});
```

### Solución Ejercicio 2

```
document.addEventListener("keydown", (e) => {  
    if (e.key === "Enter") {  
        alert("Enter pulsado");  
    }  
});
```

### Solución Ejercicio 3

```
document.addEventListener("keydown", (e) => {  
    if (e.key === "b") {  
        document.body.style.backgroundColor = "lightblue";  
    }  
});
```

## 8.3.4 Anular comportamientos predeterminados

### Explicación

Se usa:

```
event.preventDefault();
```

### Ejemplo

```
form.addEventListener("submit", (e) => {  
    e.preventDefault();  
});
```

## Ejercicios propuestos

### Ejercicio 1

Evita que un enlace navegue.

### Ejercicio 2

Evita el envío de un formulario.



### Ejercicio 3

Muestra un mensaje tras anular la acción.

### Solución Ejercicio 1

```
enlace.addEventListener("click", (e) => {  
    e.preventDefault();  
    alert("Acción cancelada");  
});
```

## 8.3.5 Cancelar propagación

### Explicación

Para evitar que el evento suba por el DOM:

```
event.stopPropagation();
```

### Ejercicios propuestos

#### Ejercicio 1

Crea dos elementos anidados y captura evento hacer click y muestra resultados por consola (efecto bubblig por defecto).

#### Ejercicio 2

Evita que el evento llegue al padre (si es pulsado el hijo).

#### Ejercicio 3

Comprueba la diferencia con y sin `stopPropagation`.

### Solucion ejercicio 2

```
hijo.addEventListener("click", (e) => {  
    e.stopPropagation();  
    alert("Solo hijo");  
});
```

## 8.3.6 Lanzar eventos

### Explicación

JavaScript permite **crear y lanzar eventos** manualmente.

```
let evento = new Event("click");  
element.dispatchEvent(evento);
```

### Ejercicios propuestos

#### Ejercicio 1

Lanza un evento `click` desde JavaScript de forma manual al iniciar el código de tu aplicación.

#### Ejercicio 2

Crea un botón que dispare otro botón (es decir, lance el evento `click` del segundo botón).

#### Ejercicio 3

Muestra un mensaje al lanzar el evento.

### Solucione ejercicio 1

```
let evento = new Event("click");  
boton.dispatchEvent(evento);
```

### Ejercicios propuestos (practicar)

1. Muestra coordenadas del ratón en tiempo real.
2. Detecta combinaciones de teclas.
3. Cancela eventos en formularios.
4. Usa `event.target` para modificar elementos.
5. Dispara eventos personalizados.

## Resumen del punto 8.3 (Objeto Evento)

Uso del objeto `event`

Coordenadas del ratón

Eventos de teclado

Cancelar acciones y propagación

Crear y lanzar eventos

## 8.4 LISTA DE TIPOS DE EVENTOS

En este apartado aprenderemos los **principales tipos de eventos** que se utilizan en JavaScript y **cuándo usar cada uno**.

Veremos ejemplos sencillos, ejercicios prácticos y soluciones.

### 8.4.1 Eventos de ratón

#### Explicación

Los **eventos de ratón** se producen cuando el usuario interactúa con el ratón.

#### Eventos más importantes

- `click` → clic simple
- `dblclick` → doble clic
- `mouseover` → ratón entra en un elemento
- `mouseout` → ratón sale del elemento
- `mousedown` → botón pulsado
- `mouseup` → botón liberado
- `mousemove` → movimiento del ratón

#### Ejemplo

```
caja.addEventListener("mouseover", () => {  
  caja.style.backgroundColor = "orange";  
});
```

#### Ejercicios propuestos

##### Ejercicio 1

Cambia el color de un `div` al pasar el ratón.

##### Ejercicio 2

Muestra un mensaje con doble clic.

##### Ejercicio 3

Cuenta cuántas veces se hace clic en un botón.

## Soluciones

```
// Ejercicio 1
caja.addEventListener("mouseover", () => {
  caja.style.backgroundColor = "lightgreen";
});

// Ejercicio 2
caja.addEventListener("dblclick", () => {
  alert("Doble clic");
});

// Ejercicio 3
let contador = 0;
boton.addEventListener("click", () => {
  contador++;
  console.log(contador);
});
```

## 8.4.2 Eventos de teclado

### Explicación

Se producen cuando el usuario pulsa una tecla.

### Eventos principales

- `keydown` → tecla pulsada
- `keyup` → tecla soltada

### Ejemplo

```
document.addEventListener("keydown", (e) => {
  console.log(e.key);
});
```

### Ejercicios propuestos

#### Ejercicio 1

Muestra la tecla pulsada.

#### Ejercicio 2

Detecta si se pulsa Escape.

#### Ejercicio 3

Cambia el color del fondo al pulsar una tecla.

## Soluciones

```
// Ejercicio 1
document.addEventListener("keydown", (e) => {
  console.log(e.key);
});

// Ejercicio 2
document.addEventListener("keydown", (e) => {
  if (e.key === "Escape") {
    alert("Escape pulsado");
  }
});

// Ejercicio 3
document.addEventListener("keydown", (e) => {
  document.body.style.backgroundColor = "lightyellow";
});
```

## 8.4.3 Eventos de movimiento en la ventana

### Explicación

Detectan cambios en la ventana del navegador.

### Eventos principales

- `resize` → cambio de tamaño
- `scroll` → desplazamiento

### Ejemplo

```
window.addEventListener("resize", () => {
  console.log("Ventana redimensionada");
});
```

### Ejercicios (8.4.3)

#### Ejercicio 1

Muestra un mensaje al cambiar el tamaño de la ventana.

#### Ejercicio 2

Muestra la posición del scroll.

#### Ejercicio 3

Cambia el color del fondo al hacer scroll.

## Soluciones

```
// Ejercicio 1
window.addEventListener("resize", () => {
  alert("Ventana cambiada");
});

// Ejercicio 2
window.addEventListener("scroll", () => {
  console.log(window.scrollY);
});

// Ejercicio 3
window.addEventListener("scroll", () => {
  document.body.style.backgroundColor = "lightblue";
});
```

## 8.4.4 Eventos sobre carga y descarga de elementos

### Explicación

Permiten saber cuándo un documento o recurso se ha cargado o se va a cerrar.

### Eventos más usados

- load → todo cargado
- DOMContentLoaded → HTML cargado
- beforeunload → antes de salir

### Ejemplo

```
document.addEventListener("DOMContentLoaded", () => {
  console.log("Documento listo");
});
```

### Ejercicios propuestos

#### Ejercicio 1

Muestra un mensaje cuando la página esté lista.

#### Ejercicio 2

Muestra un mensaje cuando una imagen se cargue.

#### Ejercicio 3

Detecta cuando el usuario intenta salir.

## Soluciones

```
// Ejercicio 1
document.addEventListener("DOMContentLoaded", () => {
  alert("Página cargada");
});

// Ejercicio 2
imagen.addEventListener("load", () => {
  console.log("Imagen cargada");
});

// Ejercicio 3
window.addEventListener("beforeunload", () => {
  console.log("Saliendo...");
});
```

## 8.4.5 Otros eventos

### Explicación

Eventos adicionales muy usados:

- `focus` → elemento recibe foco
- `blur` → pierde foco
- `change` → cambia valor
- `input` → entrada de datos

### Ejemplo

```
input.addEventListener("focus", () => {
  input.style.backgroundColor = "lightyellow";
});
```

## Ejercicios propuestos

### Ejercicio 1

Cambia el color de un input al recibir foco.

### Ejercicio 2

Detecta cuando se cambia un valor.

### Ejercicio 3

Muestra lo que el usuario escribe.

## Soluciones

```
// Ejercicio 1
input.addEventListener("focus", () => {
  input.style.backgroundColor = "lightgreen";
});

// Ejercicio 2
input.addEventListener("change", () => {
  console.log("Valor cambiado");
});

// Ejercicio 3
input.addEventListener("input", () => {
  console.log(input.value);
});
```

## Ejercicios propuestos (practicar)

1. Cambia estilos usando varios eventos de ratón.
2. Crea atajos de teclado.
3. Detecta scroll y muestra porcentaje.
4. Muestra mensajes al cargar recursos.
5. Valida entradas usando `focus` y `blur`.

## Resumen del punto 8.4 (Lista tipo de Eventos)

Eventos de ratón

Eventos de teclado

Eventos de ventana

Eventos de carga

Eventos de formulario y foco



## 8.5 FORMULARIOS

Los **formularios** permiten recoger información del usuario.

En JavaScript, los formularios y sus controles se tratan como **objetos del DOM**, por lo que podemos acceder a ellos, leer valores y reaccionar a eventos.

### 8.5.1 El formulario como objeto del DOM

#### Explicación

Un formulario se puede acceder:

- Por `id`
- Por `name`
- A través de `document.forms` (es un array con índices)

#### Ejemplo

```
<form id="miFormulario">
  <input type="text" name="nombre">
</form>
let form = document.getElementById("miFormulario");
```

También:

```
let form = document.forms[0];
```

## Ejemplo de formulario y cómo acceder a los campos del formulario.

```
<form name="registro" id="formRegistro">
  <label for="nombreUsuario">Nombre:</label>
  <input type="text" name="nombreUsuario" id="nombreUsuario"><br><br>

  <label for="emailUsuario">Email:</label>
  <input type="email" name="emailUsuario" id="emailUsuario"><br><br>

  <button type="submit">Enviar</button>
</form>

<script>
  document.getElementById('formRegistro').addEventListener('submit', function(event)
  {
    event.preventDefault(); // Evita que el formulario se envíe normalmente

    const form = document.forms['registro']; // Accede al formulario por su 'name'

    // Accede a los campos por su 'name'
    const nombre = form.elements['nombreUsuario'].value;
    const email = form.elements['emailUsuario'].value;

    console.log("Nombre:", nombre);
    console.log("Email:", email);
  });
</script>
```

## Ejercicios propuestos

### Ejercicio 1

Accede a un formulario por su `id`.

### Ejercicio 2

Accede a un input usando su atributo `name`.

### Ejercicio 3

Muestra en consola el número de formularios de la página.

## Soluciones

```
// Ejercicio 1
let form = document.getElementById("miFormulario");

// Ejercicio 2
let nombre = form.elements["nombre"];

// Ejercicio 3
console.log(document.forms.length);
```

## 8.5.2 Eventos de formulario

### Explicación

Eventos más importantes:

- `submit` → envío
- `reset` → reinicio
- `change` → cambio
- `input` → escritura

### Ejemplo

```
form.addEventListener("submit", (e) => {
  e.preventDefault();
  alert("Formulario enviado");
});
```

### Ejercicios propuestos

#### Ejercicio 1

Evita el envío del formulario.

#### Ejercicio 2

Muestra un mensaje al enviar.

#### Ejercicio 3

Detecta cambios en un input.

## Soluciones

```
// Ejercicio 1 y 2
form.addEventListener("submit", (e) => {
  e.preventDefault();
  alert("Envío cancelado");
});

// Ejercicio 3
input.addEventListener("change", () => {
  console.log("Valor cambiado");
});
```

## 8.5.3 Propiedades de los controles

### Explicación

Propiedades más usadas:

- `value` → valor
- `checked` → checkbox/radio
- `disabled` → deshabilitar
- `selectedIndex` → select

### Ejemplo

```
console.log(input.value);
```

### Ejercicios propuestos

#### Ejercicio 1

Obtén el valor de un `input`.

#### Ejercicio 2

Comprueba si un checkbox está marcado.

#### Ejercicio 3

Obtén la opción seleccionada de un `select`.

## Soluciones

```
// Ejercicio 1
console.log(input.value);

// Ejercicio 2
if (checkbox.checked) {
  console.log("Marcado");
}
```

```
// Ejercicio 3
console.log(select.selectedIndex);
```

## 8.5.4 Métodos de los controles

### Explicación

Métodos más importantes:

- `focus()` → poner foco
- `blur()` → quitar foco
- `reset()` → reiniciar
- `submit()` → enviar

### Ejemplo

```
input.focus();
```

### Ejercicios propuestos

#### Ejercicio 1

Coloca el foco en un input al cargar la página.

#### Ejercicio 2

Reinicia el formulario con un botón.

#### Ejercicio 3

Envía el formulario desde JavaScript.

### Soluciones

```
// Ejercicio 1
window.addEventListener("load", () => {
  input.focus();
});
```

```
// Ejercicio 2
botonReset.addEventListener("click", () => {
  form.reset();
});
```

```
// Ejercicio 3
form.submit();
```

## Ejercicios propuestos

1. Valida que los campos no estén vacíos.
2. Muestra los datos introducidos.
3. Bloquea el envío si hay errores.
4. Cambia estilos según validación.
5. Crea un formulario interactivo completo.

### Solución ejemplo (Ejercicio 1)

```
form.addEventListener("submit", (e) => {  
  if (input.value === "") {  
    e.preventDefault();  
    alert("Campo obligatorio");  
  }  
});
```

## Resumen del punto 8.5 (Formularios)

- ✓ Formularios como objetos DOM
- ✓ Eventos de formulario
- ✓ Propiedades de controles
- ✓ Métodos útiles
- ✓ Validación básica